

# Software Requirements Specification (SRS)

**Project: ConnectSphere -- Core Social Media & Real-Time Messaging Engine**

## 1. System Overview

The system is a high-performance, modular backend engine designed to support a scalable social networking platform. It handles user identity, social graphing, rich-media content distribution, dynamic feed aggregation, and zero-drop, real-time messaging with granular delivery tracking.

## 2. Functional Requirements (FR)

Functional requirements describe the specific capabilities and behaviours the system must execute.

### 2.1 Identity & Profile Management (auth & profile)

- **FR-1.1:** The system must allow users to register with a unique email, username, and encrypted password.
- **FR-1.2:** The system must authenticate users, issue a stateless JSON Web Token (JWT), and securely transport it via an 'HttpOnly', Secure cookie.
- **FR-1.3:** The system must provide a global authentication middleware to protect private endpoints.
- **FR-1.4:** The system must allow users to create and update profiles containing separate metadata (display names, bios, and avatar URLs) isolated from core credential records.
- **FR-1.5:** Profile updates must instantly propagate across all contextual lookups (e.g., changing a name updates the display name in an active inbox).

### 2.2 Social Graph & Content Distribution (follow & post)

- **FR-2.1:** The system must support directional social relationships (User A follows User B asynchronously).
- **FR-2.2:** The system must allow users to create posts containing textual content and optional rich-media assets (images).
- **FR-2.3:** The system must expose middleware to handle incoming multipart file streams, save assets locally, and generate a accessible static public URL.
- **FR-2.4:** The system must allow targeted retrieval of single post payloads or an individual user's chronological timeline.

### 2.3 Feed Generation (feed)

- **FR-3.1:** The system must dynamically compile a personalized activity feed for a requesting user, aggregating posts from all followed accounts alongside the user's own posts.
- **FR-3.2:** The aggregated feed must be ordered reverse-chronologically based on creation timestamps.
- **FR-3.3:** The feed must support pagination utilizing a cursor-based strategy (timestamp markers) to allow infinite scrolling.

## 2.4 Real-Time Chat Engine (chat)

- **FR-4.1:** The system must support Direct Messages (DMs) between two users and Group Chats among multiple participants.
- **FR-4.2:** Every sent message must be durably written to the relational database before any network broadcasting is attempted.
- **FR-4.3:** The system must seamlessly resolve DM conversation titles dynamically to match the `display_name` of the *other* participant.
- **FR-4.4:** The system must evaluate active connection pools and broadcast messages instantly via WebSockets to online participants.
- **FR-4.5:** The system must support an HTTP retrieval route for historical chat logs, protected by strict room-membership checks.
- **FR-4.6:** The system must capture and emit real-time message state mutations, tracing receipts through three discrete phases: *Sent*, *Delivered* (received by device), and *Read* (viewed by user).
- **FR-4.7:** The system must allow users to dynamically mutate group states by adding members, removing members, or voluntarily leaving threads, immediately updating the real-time broadcast paths.

## 3. Non-Functional Requirements (NFR)

Non-functional requirements specify the operational criteria, constraints, and quality attributes used to judge the operation of a system.

| Category | Requirement ID | Specification Description                                                                                                                                                               |
|----------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security | NFR-1.1        | <b>Zero-Trust Access Control:</b> All cross-domain actions and stateful WebSocket handshakes must require validation of a cryptographically signed JWT.                                 |
|          | NFR-1.2        | <b>Data Isolation:</b> Former group conversation participants must be strictly locked out from fetching historical records or receiving future broadcast events instantly upon leaving. |
|          | NFR-1.3        | <b>Credential Protection:</b> Plaintext passwords must never be stored; they must be hashed using a computationally secure algorithm (e.g., bcrypt) prior to database persistence.      |

| Category    | Requirement ID | Specification Description                                                                                                                                                                                                                    |
|-------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Performance | NFR-2.1        | <b>Low-Latency Feed Read:</b> Feed requests must serve within < 200ms by prioritizing in-memory caching layers (Redis) over cold database reads.                                                                                             |
|             | NFR-2.2        | <b>Real-Time Delivery:</b> WebSocket message fan-out to online users must achieve a transmission latency of < 100ms from the moment database write confirmation occurs.                                                                      |
|             | NFR-2.3        | <b>Connection Pooling:</b> The system must enforce active database connection pooling to handle up to 20 concurrent connections per node instance without starvation.                                                                        |
| Scalability | NFR-3.1        | <b>Pagination Efficiency:</b> The system must ban offset-based pagination (LIMIT X OFFSET Y) on infinite lists, enforcing cursor-based pagination to maintain a flat $O(1)$ lookup complexity regardless of data depth.                      |
|             | NFR-3.2        | <b>Database Normalization:</b> Redundant data strings (like usernames) must not be duplicated across operational transaction tables to minimize memory footprints and ensure data consistency.                                               |
| Reliability | NFR-4.1        | <b>Durable-First Transactions:</b> In-flight network message packet loss must not result in data loss; messages must successfully hit PostgreSQL disk storage before transport triggers.                                                     |
|             | NFR-4.2        | <b>Atomic Operations:</b> Critical multi-step database mutations (such as creating a group and unnesting its list of initial participants) must execute inside a strict SQL Transaction (BEGIN / COMMIT) to avoid orphan records on failure. |

#### 4. Technical Stack & Environment Constraints

- **Language Runtime:** TypeScript compiled to clean ECMAScript modules running on Node.js v20.x or higher.
- **Web Framework:** Express.js using non-blocking asynchronous routing handler design.
- **Primary Database:** PostgreSQL (utilizing advanced features like transactional isolation levels, dynamic CASE conditionals, array UNNEST mutations, and LATERAL joins).
- **Caching & State Store:** Redis Server for high-performance volatile state mapping and timeline storage.
- **Real-Time Gateway:** Socket.IO engine bound to the primary HTTP transport server wrapper to enforce single-port resource optimization.